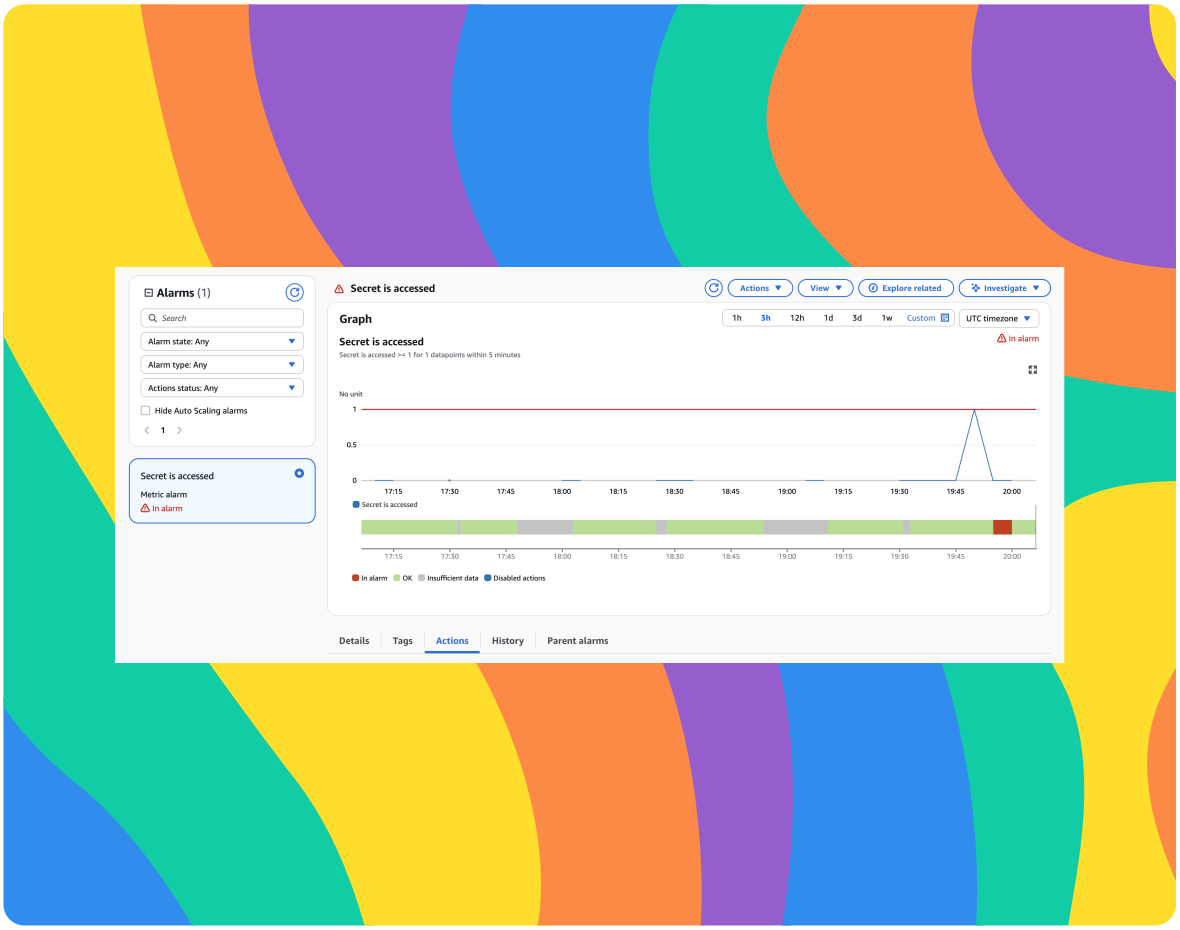
nextwork.org

Build a Security Monitoring System

CH Arpita Chowdhury



CH
Arpita Chowdhury
NextWork Student
nextwork.org

Introducing Today's Project!

In this project, I will demonstrate how to securely store sensitive data, enable detailed logging, and build a real-time monitoring and alerting system in AWS using Secrets Manager, CloudTrail, CloudWatch, and SNS. I’m doing this project to learn how to detect and respond to sensitive access events, understand how AWS logs and monitoring services work together, and gain hands-on experience building an automated security alert pipeline that reflects real-world cloud security best practices.

Tools and concepts

The services I used were AWS Secrets Manager, AWS CloudTrail, Amazon CloudWatch (Logs, Metrics, and Alarms), Amazon SNS, AWS IAM, and AWS CLI. Key concepts I learnt include securely storing and accessing secrets, auditing API activity with CloudTrail, streaming logs into CloudWatch Logs, creating metric filters to detect sensitive actions like GetSecretValue, configuring CloudWatch alarms based on custom metrics, integrating SNS for alerting, and troubleshooting end-to-end monitoring pipelines when notifications don’t behave as expected.

Project reflection

This project took me approximately 3 hours.

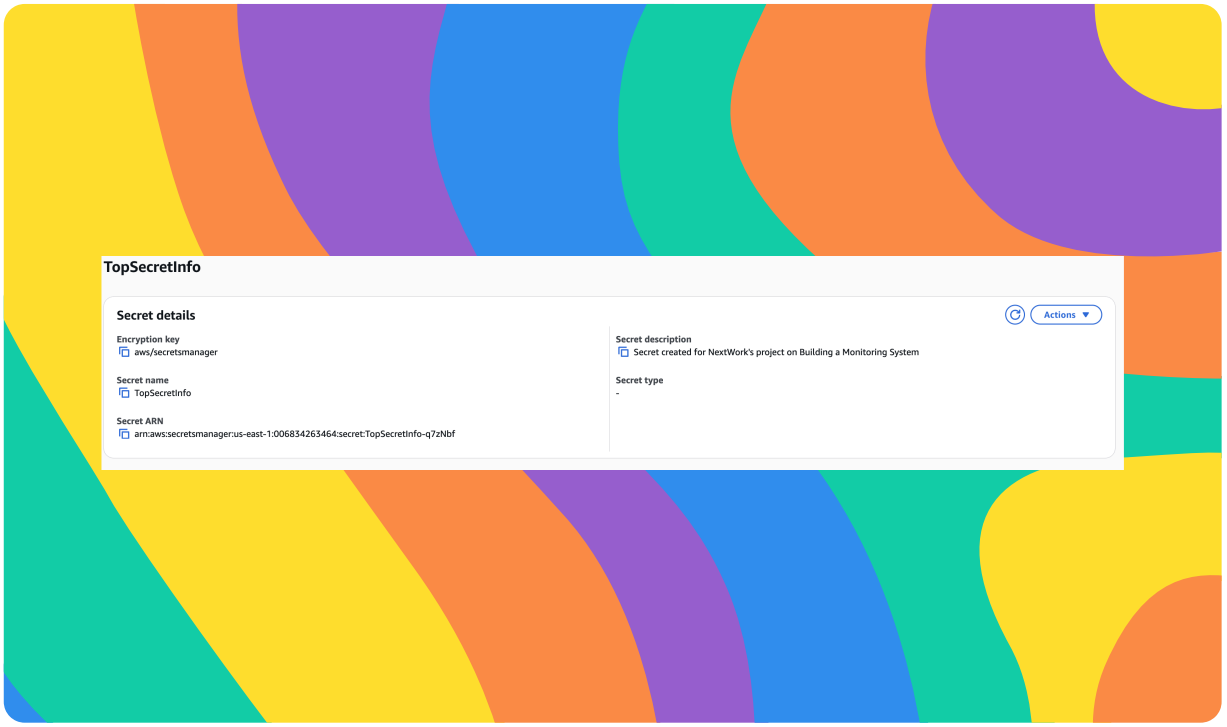
The most challenging part was troubleshooting the end-to-end monitoring flow, especially figuring out why the CloudWatch alarm was triggering but the SNS email notification was not being delivered, and carefully verifying each component (CloudTrail, CloudWatch Logs, metric filters, alarms, and SNS subscriptions). It was most rewarding to finally validating that the alarm logic worked correctly and understanding how real-world security monitoring and alerting pipelines are built and debugged in AWS, which made the project feel very practical and job-ready.

CH
Arpita Chowdhury
NextWork Student

Create a Secret

Secrets Manager is an AWS service that securely stores, manages, and retrieves sensitive information such as passwords, API keys, database credentials, and tokens. You could use Secrets Manager to avoid hard-coding secrets in applications, rotate credentials automatically, control access using IAM policies, and audit when secrets are accessed helping improve security and reduce the risk of credential exposure.

To set up for my project, I created a secret called TopSecretInfo that contains sensitive information stored securely in AWS Secrets Manager, which I will later access to generate logs and test my monitoring and alerting setup.



CH
Arpita Chowdhury
NextWork Student
nextwork.org

Set Up CloudTrail

CloudTrail is an AWS service that records and logs all API activity and actions taken in an AWS account, including who did what, when, and from where. I set up a trail to capture and store logs of sensitive actions such as accessing secrets in AWS Secrets Manager so I can monitor activity, analyze events, and trigger alerts when important or suspicious actions occur.

CloudTrail events include types like management events, which track actions taken on AWS resources (such as creating, updating, or accessing services like Secrets Manager), data events, which record high-volume activities on resources like S3 objects or Lambda function invocations, and insight events, which detect unusual or anomalous API activity patterns that may indicate security issues or operational problems.

Read vs Write Activity

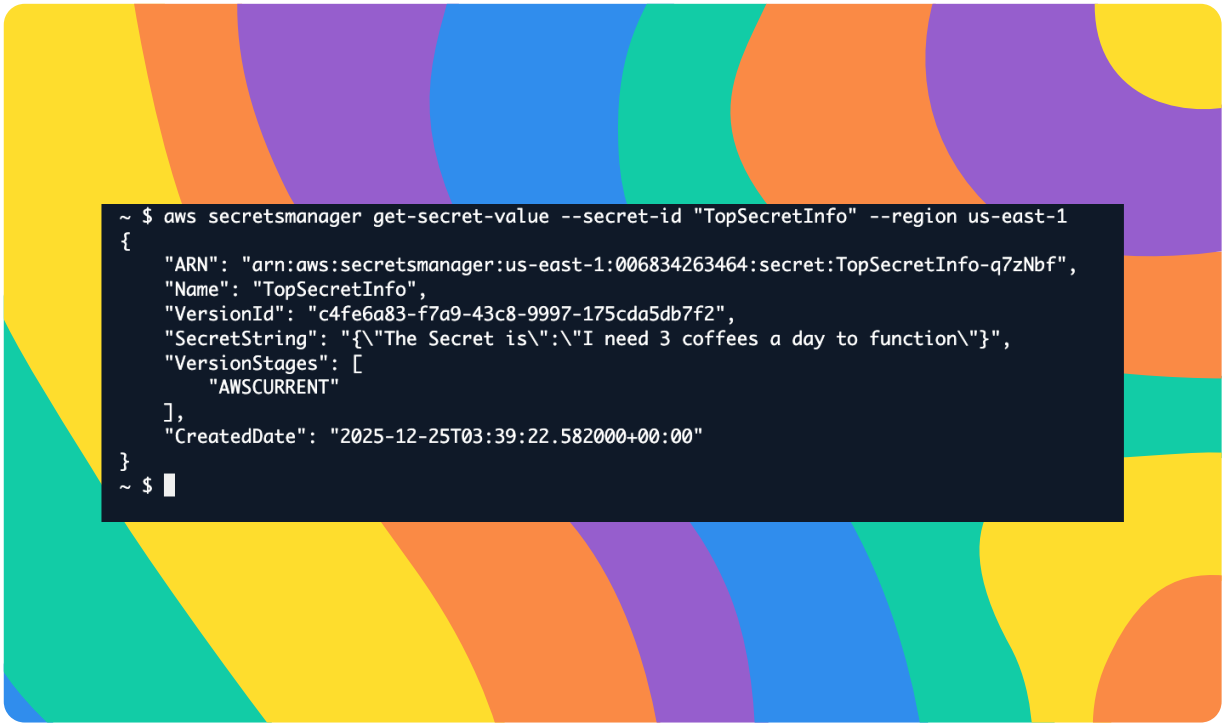
Read API activity involves viewing or retrieving information about AWS resources—like GetSecretValue, DescribeSecret, or ListSecrets in Secrets Manager. Write API activity involves creating, changing, or deleting resources or configurations—like CreateSecret, PutSecretValue, UpdateSecret, or DeleteSecret. For this project, we need Read API activity logging, because the goal is to detect and alert when someone accesses (reads) the secret, which is the sensitive event we want CloudTrail and CloudWatch to capture.

CH
Arpita Chowdhury
NextWork Student

Verifying CloudTrail

I retrieved the secret in two ways: first through the AWS Management Console by clicking “Retrieve secret value” in Secrets Manager, and second using the AWS CLI with the aws secretsmanager get-secret-value command, which allowed me to generate CloudTrail events for both console-based and programmatic access.

To analyze my CloudTrail events, I visited the CloudTrail Event history in the AWS Management Console. I found a recorded GetSecretValue event showing that my secret was accessed, including details like the time, source, and API call used. This tells me that CloudTrail is correctly logging secret access activity and that my environment is ready for monitoring and alerting on sensitive actions.



CH
Arpita Chowdhury
NextWork Student
nextwork.org

CloudWatch Metrics

CloudWatch Logs is an AWS service that collects, stores, and lets you search and analyze log data from AWS services and applications. It's important for monitoring because it enables real-time visibility into activity, allows you to create metrics and alarms from log events, and helps detect, investigate, and respond to sensitive or suspicious actions such as access to secrets as soon as they occur.

CloudTrail's Event History is useful for quickly viewing and manually inspecting recent API activity to understand what actions occurred in your account. CloudWatch Logs are better for long-term storage, advanced searching, creating metrics and alarms, and building automated monitoring and alerting systems based on specific log patterns, such as access to sensitive secrets.

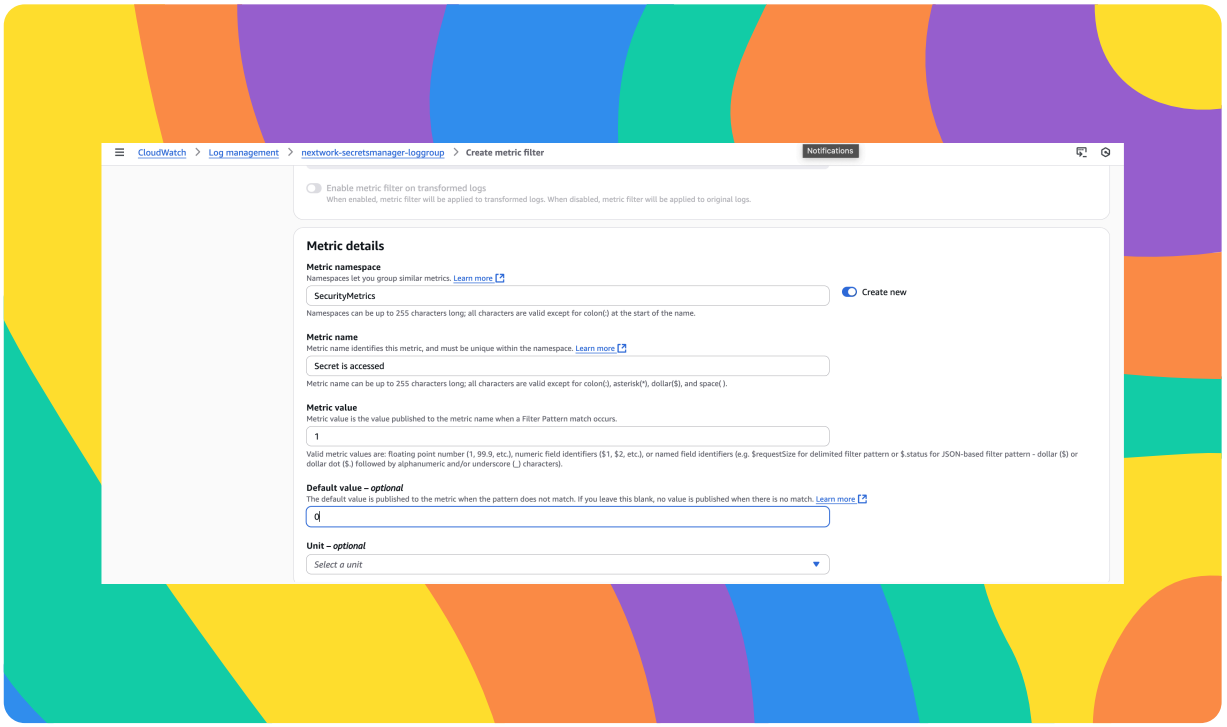
A CloudWatch metric is a numerical data point that represents the occurrence or measurement of a specific event or activity over time. When setting up a metric, the metric value represents how many times a matching event occurs (for example, each time a GetSecretValue API call appears in the logs). The default value is used when no matching log events are found during a given time period, ensuring the metric still reports data consistently even when the monitored activity does not occur.

CH

Arpita Chowdhury

NextWork Student

nextwork.org



CH
Arpita Chowdhury
NextWork Student
nextwork.org

CloudWatch Alarm

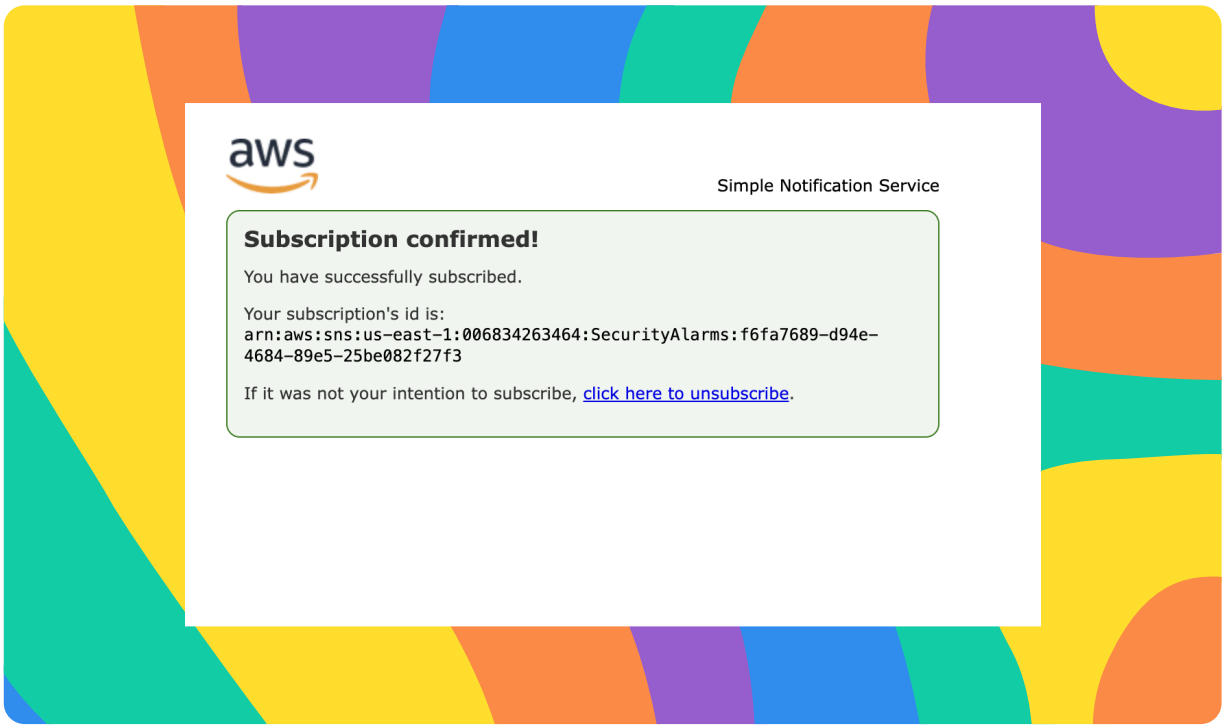
A CloudWatch alarm is a monitoring tool that watches a CloudWatch metric and automatically triggers an action when the metric crosses a defined threshold. I set my CloudWatch alarm threshold to 1 so the alarm will trigger when the secret is accessed at least once, immediately alerting me that sensitive data has been read.

An SNS topic is a communication channel in AWS that allows messages to be published and delivered to multiple subscribers. My SNS topic is set up to send an email notification whenever the CloudWatch alarm is triggered, ensuring I’m alerted immediately when the secret is accessed.

AWS requires email confirmation because it needs to verify that the email address owner has explicitly agreed to receive notifications. This helps prevent spam, accidental subscriptions, and unauthorized use of someone else’s email address, ensuring alerts are delivered only to intended and approved recipients.

CH

Arpita Chowdhury
NextWork Student



CH
Arpita Chowdhury
NextWork Student
nextwork.org

Troubleshooting Notification Errors

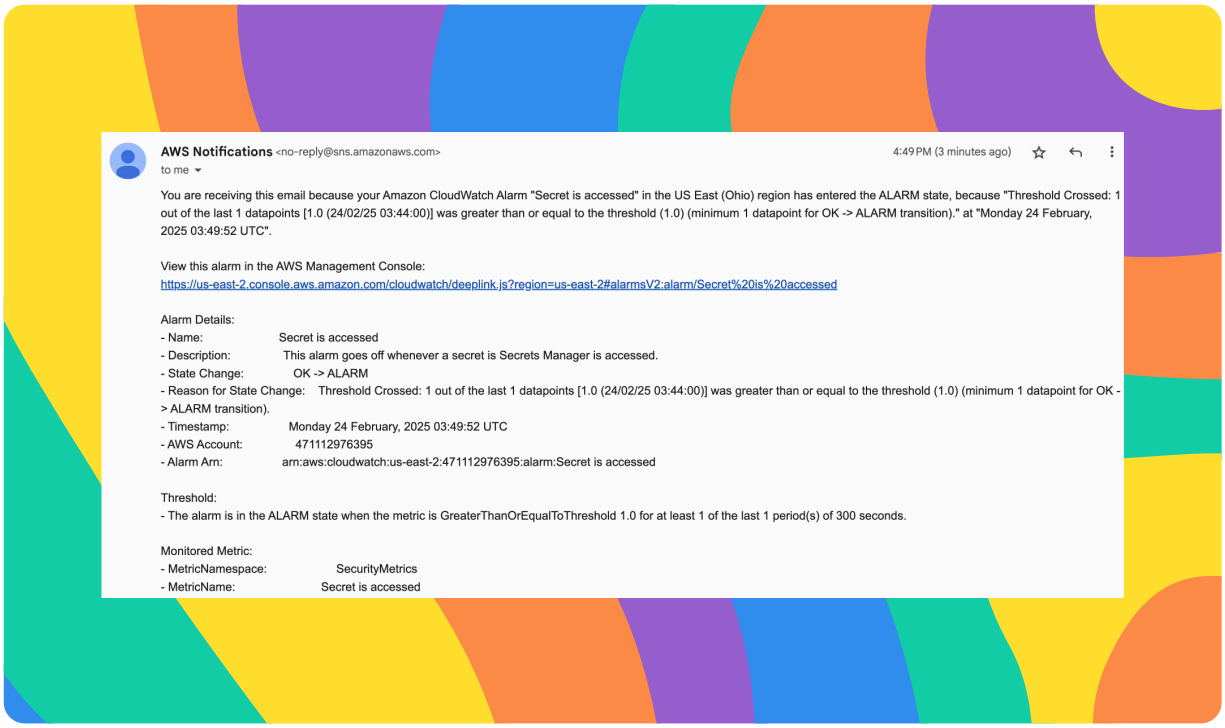
To test my monitoring system, I retrieved the secret value again and reviewed the CloudTrail logs, CloudWatch metric filters, alarm state, and SNS subscription settings. The results were that the secret access was successfully logged, but the alarm did not trigger immediately, helping me identify configuration or timing issues and confirm where adjustments were needed to get the notification system working correctly.

The 5 ways I did troubleshooting for this error were: Checked whether CloudTrail recorded the GetSecretValue event by reviewing CloudTrail Event History to confirm that secret access was actually being logged. Verified that CloudTrail was configured to send logs to CloudWatch Logs, including checking the log group, IAM role permissions, and trail settings. Reviewed the CloudWatch metric filter configuration to ensure it was matching the correct log group, event name (GetSecretValue), and JSON pattern. Checked the CloudWatch Alarm configuration and state to confirm it was linked to the correct metric, had the right threshold, and was set to trigger an action. Confirmed the SNS setup, including email subscription confirmation and SNS permissions, to ensure notifications could be successfully delivered.

I initially didn’t receive an email before because the monitoring chain wasn’t fully connected—CloudTrail events weren’t successfully flowing into CloudWatch Logs (or the metric filter wasn’t matching the GetSecretValue event), so the SecretIsAccessed metric never increased and the alarm never triggered SNS. The key solution was fixing the CloudTrail → CloudWatch Logs integration and correcting the metric filter pattern/log group so GetSecretValue events were detected, which allowed the metric to update, the alarm to enter ALARM state, and SNS to finally send the email notification.

Success!

To validate the monitoring and alerting system, I checked the CloudWatch alarm history and SNS action logs to confirm that the alarm transitioned from OK to ALARM and that the notification action was successfully executed. I received confirmation within AWS that the SNS notification was sent, validating that the end-to-end pipeline from secret access to alert execution was working as designed.



CH
Arpita Chowdhury
NextWork Student
nextwork.org

Comparing CloudWatch with CloudTrail Notifications

In a project extension, I updated my CloudTrail configurations to ensure events were sent to CloudWatch Logs and included the specific API activity I wanted to monitor because this enabled real-time detection and alerting for sensitive actions, such as secret access, instead of relying only on manual log review.

After enabling CloudTrail SNS notifications, my inbox received an email alert each time the secret was accessed, confirming that the monitoring and alerting pipeline was working correctly. In terms of the usefulness of these emails, I thought they were very effective for immediate awareness, as they provide near real-time visibility into sensitive actions and would allow a quick response if the access were unauthorized.

CH

Arpita Chowdhury
NextWork Student

The place to learn & showcase your skills

Check out nextwork.org for more projects